

Table 3: Test accuracy (%) on our 20 Tasks for various methods (1000 training examples each). Our proposed extensions to MemNNs are in columns 5-9: with adaptive memory (AM), N -grams (NG), nonlinear matching function (NL), and combinations thereof. Bold numbers indicate tasks where our extensions achieve $\geq 95\%$ accuracy but the original MemNN model of Weston et al. (2014) did not. The last two columns (10-11) give extra analysis of the $\text{MemNN}^{\text{AM} + \text{NG} + \text{NL}}$ method. Column 10 gives the amount of training data for each task needed to obtain $\geq 95\%$ accuracy, or *FAIL* if this is not achievable with 1000 training examples. The final column gives the accuracy when training on all data at once, rather than separately.

TASK	Weakly Supervised		Uses External Resources	Strong Supervision (using supporting facts)						
	N -gram Classifier	LSTM	Structured STY <i>Coref + Set features</i>	MemNN <i>Weston et al. (2014)</i>	MemNN <i>ADAPTIVE MEMORY</i>	MemNN <i>AM + k-CRAMS</i>	MemNN <i>AM + NONLINEAR</i>	MemNN <i>AM + NG + NL</i>	No. of ex. req. ≥ 95	Multi-task training
1 - Single Supporting Fact	36	50	99	100	100	100	100	100	250 ex.	100
2 - Two Supporting Facts	2	20	74	100	100	100	100	100	500 ex.	100
3 - Three Supporting Facts	7	20	17	20	100	99	100	100	500 ex.	98
4 - Two Arg. Relations	50	61	98	71	69	100	73	100	500 ex.	80
5 - Three Arg. Relations	20	70	83	83	83	86	86	98	1000 ex.	99
6 - Yes/No Questions	49	48	99	47	52	53	100	100	500 ex.	100
7 - Counting	52	49	69	68	78	86	83	85	FAIL	86
8 - Lists/Sets	40	45	70	77	90	88	94	91	FAIL	93
9 - Simple Negation	62	64	100	65	71	63	100	100	500 ex.	100
10 - Indefinite Knowledge	45	44	99	59	57	54	97	98	1000 ex.	98
11 - Basic Coreference	29	72	100	100	100	100	100	100	250 ex.	100
12 - Conjunction	9	74	96	100	100	100	100	100	250 ex.	100
13 - Compound Coref.	26	94	99	100	100	100	100	100	250 ex.	100
14 - Time Reasoning	19	27	99	99	100	99	100	99	500 ex.	99
15 - Basic Deduction	20	21	96	74	73	100	77	100	100 ex.	100
16 - Basic Induction	43	23	24	27	100	100	100	100	100 ex.	94
17 - Positional Reasoning	46	51	61	54	46	49	57	65	FAIL	72
18 - Size Reasoning	52	52	62	57	50	74	54	95	1000 ex.	93
19 - Path Finding	0	8	49	0	9	3	15	36	FAIL	19
20 - Agent's Motivations	76	91	95	100	100	100	100	100	250 ex.	100
Mean Performance	34	49	79	75	79	83	87	93		92

Methods The N -gram classifier baseline is inspired by the baselines in Richardson et al. (2013) but applied to the case of producing a 1-word answer rather than a multiple choice question: we construct a bag-of- N -grams for all sentences in the story that share at least one word with the question, and then learn a linear classifier to predict the answer using those features⁴.

LSTMs are a popular method for sequence prediction (Sutskever et al., 2014) and outperform standard RNNs (Recurrent Neural Networks) for tasks similar to ours (Weston et al., 2014). They work by reading the story until the point they reach a question and then have to output an answer. Note that they are weakly supervised by answers only, and are hence at a disadvantage compared to strongly supervised methods or methods that use external resources.

MemNNs (Weston et al., 2014) are a recently proposed class of models that have been shown to perform well at QA. They work by a “controller” neural network performing inference over the stored memories that consist of the previous statements in the story. The original proposed model performs 2 hops of inference: finding the first supporting fact with the maximum match score with the question, and then the second supporting fact with the maximum match score with both the question and the first fact that was found. The matching function consists of mapping the bag-of-words for the question and facts into an embedding space by summing word embeddings. The word embeddings are learnt using strong supervision to optimize the QA task. After finding supporting facts, a final ranking is performed to rank possible responses (answer words) given those facts. We also consider some extensions to this model:

- **Adaptive memories** performing a variable number of hops rather than 2, the model is trained to predict a hop or the special “STOP” class. A similar procedure can be applied to output multiple tokens as well.

⁴Constructing N -grams from all sentences rather than using the filtered set gave worse results.

- ***N*-grams** We tried using a bag of 3-grams rather than a bag-of-words to represent the text. In both cases the first step of the MemNN is to convert these into vectorial embeddings.
- **Nonlinearity** We apply a classical 2-layer neural network with tanh nonlinearity in the matching function.

More details of these variants is given in Sec A of the appendix.

Finally, we built a classical cascade NLP system baseline using a structured support vector machine (SVM), which incorporates coreference resolution and semantic role labeling (SRL) preprocessing steps, which are themselves trained on large amounts of costly labeled data. The Stanford coreference system (Raghunathan et al., 2010) and the SENNA semantic role labeling (SRL) system (Collobert et al., 2011) are used to build features for the input to the SVM, trained with strong supervision to find the supporting facts, e.g. features based on words, word pairs, and the SRL verb and verb-argument pairs. After finding the supporting facts, we build a similar structured SVM for the response stage, with features tuned for that goal as well. More details are in Sec. B of the appendix.

Learning rates and other hyperparameters for all methods are chosen using the training set. The summary of our experimental results on the tasks is given in Table 3. We give results for each of the 20 tasks separately, as well as mean performance and number of failed tasks in the final two rows.

Results Standard MemNNs generally outperform the *N*-gram and LSTM baselines, which is consistent with the results in Weston et al. (2014). However they still “fail” at a number of tasks; that is, test accuracy is less than 95%. Some of these failures are expected due to insufficient modeling power as described in more detail in Sec. A.1, e.g. $k = 2$ facts, single word answers and bag-of-words do not succeed on tasks 3, 4, 5, 7, 8 and 18. However, there were also failures on tasks we did not at first expect, for example yes/no questions (6) and indefinite knowledge (10). Given hindsight, we realize that the linear scoring function of standard MemNNs cannot model the match between query, supporting fact and a yes/no answer as this requires three-way interactions.

Columns 5-9 of Table 3 give the results for our MemNN extensions: adaptive memories (AM), *N*-grams (NG) and nonlinearities (NL), plus combinations thereof. The adaptive approach gives a straight-forward improvement in tasks 3 and 16 because they both require more than two supporting facts, and also gives (small) improvements in 8 and 19 because they require multi-word outputs (but still remain difficult). We hence use the AM model in combination with all our other extensions in the subsequent experiments.

MemNNs with *N*-gram modeling yield clear improvements when word order matters, e.g. tasks 4 and 15. However, *N*-grams do not seem to be a substitute for nonlinearities in the embedding function as the NL model outperforms *N*-grams on average, especially in the yes/no (6) and indefinite tasks (10), as explained before. On the other hand, the NL method cannot model word order and so fails e.g., on task 4. The obvious step is thus to combine these complimentary approaches: indeed AM+NG+NL (column 9) gives improved results over both, with a total of 9 tasks that have been upgraded from failure to success compared to the original MemNN model.

The structured SVM, despite having access to external resources, does not perform better, still failing at 9 tasks. It does perform better than vanilla MemNNs (without extensions) on tasks 6, 9 and 10 where the hand-built feature conjunctions capture the necessary nonlinearities. However, compared to MemNN (AM+NG+NL) it seems to do significantly worse on tasks requiring three (and sometimes, two) supporting facts (e.g. tasks 3, 16 and 2) presumably as ranking over so many possibilities introduces more mistakes. However, its non-greedy search does seem to help on other tasks, such as path finding (task 19) where search is very important. Since it relies on external resources specifically designed for English, it is unsure that it would perform as well on other languages, like Hindi, where such external resources might be of worse quality.

The final two columns (10-11) give further analysis of the AM+NG+NL MemNN method. The second to last column (10) shows the minimum number of training examples required to achieve $\geq 95\%$ accuracy, or *FAIL* if this is not achieved with 1000 examples. This is important as it is not only desirable to perform well on a task, but also using the fewest number of examples (to generalize well, quickly). Most succeeding tasks require 100-500 examples. Task 8 requires 5000 examples and 7 requires 10000, hence they are labeled as *FAIL*. The latter task can presumably be solved by adding all the times an object is picked up, and subtracting the times it is dropped, which seems

possible for an MemNN, but it does not do perfectly. Two tasks, positional reasoning 17 and path finding 19 cannot be solved even with 10000 examples, it seems those (and indeed more advanced forms of induction and deduction, which we plan to build) require a general search algorithm to be built into the inference procedure, which MemNN (and the other approaches tried) are lacking.

The last column shows the performance of AM+NG+NL MemNNs when training on *all* the tasks jointly, rather than just on a single one. The performance is generally encouragingly similar, showing such a model can learn many aspects of text understanding and reasoning simultaneously. The main issues are that these models still fail on several of the tasks, and use a far stronger form of supervision (using supporting facts) than is typically realistic.

6 DISCUSSION

A prerequisite set We developed a set of tasks that we believe are a prerequisite to full language understanding and reasoning. While any learner that can solve these tasks is not necessarily close to full reasoning, if a learner fails on any of our tasks then there are likely real-world tasks that it will fail on too (i.e., real-world tasks that require the same kind of reasoning). Even if the situations and the language of the tasks are artificial, we believe that the mechanisms required to *learn* how to solve them are part of the key towards text understanding and reasoning.

A flexible framework This set of tasks is not a definitive set. The purpose of a simulation-based approach is to provide flexibility and control of the tasks' construction. We grounded the tasks into language because it is then easier to understand the usefulness of the tasks and to interpret their results. However, our primary goal is to find models able to learn to detect and combine patterns in symbolic sequences. One might even want to decrease the intrinsic difficulty by removing any lexical variability and ambiguity and reason only over bare symbols, stripped down from their linguistic meaning. One could also decorrelate the long-term memory from the reasoning capabilities of systems by, for instance, arranging the supporting facts closer to the questions. In the opposing view, one could instead want to transform the tasks into more realistic stories using annotators or more complex grammars. The set of 20 tasks presented here is a subset of what can be achieved with a simulation. We chose them because they offer a variety of skills that we would like a text reasoning model to have, but we hope researchers from the community will develop more tasks of varying complexity in order to develop and analyze models that try to solve them. Transfer learning across tasks is also a very important goal, beyond the scope of this paper. We have thus made the simulator and code for the tasks publicly available for those purposes.

Testing learning methods Our tasks are designed as a test-bed for learning methods: we provide training and test sets because we intend to evaluate the capability of models to discover how to reason from patterns hidden within them. It could be tempting to hand-code solutions for them or to use existing large-scale QA systems like Cyc (Curtis et al., 2005). They might succeed at solving them, even if our structured SVM results (a cascaded NLP system with hand-built features) show that this is not straightforward; however this is not the tasks' purpose since those approaches would not be learning to solve them. Our experiments show that some existing machine learning methods are successful on some of the tasks, in particular Memory Networks, for which we introduced some useful extensions (in Sec. A). However, those models still fail on several of the tasks, and use a far stronger form of supervision (using supporting facts) than is typically realistic.

These datasets are not yet solved. Future research should aim to minimize the amount of required supervision, as well as the number of training examples needed to solve a new task, to move closer to the task transfer capabilities of humans. That is, in the weakly supervised case with only 1000 training examples or less there is no known general (i.e. non-hand engineered) method that solves the tasks. Further, importantly, our hope is that a feedback loop of developing more challenging tasks, and then algorithms that can solve them, leads us to fruitful research directions.

Note that these tasks are not a substitute for real data, but should complement them, especially when developing and analysing algorithms. There are many complementary real-world datasets, see for example Hermann et al. (2015); Bordes et al. (2015); Hill et al. (2015). That is, even if a method works well on our 20 tasks, it should be shown to be useful on real data as well.